

Handoff — Modal templates roadmap

Last updated: 2026-06-01. Sprint 0 + Sprint 1 + Sprint 2 + **Sprint 3 SHIPPED**.

Live AI render is ON (`LIVE_AI_RENDER = true`). 3-level hierarchical template posterior, Component Analysis Modal Design column, Pro device-lift upsell, and the dev-data poisoning guard all landed. See Sprint 3 status. Remaining: a real-theme eyeball of the live AI path + the 17-factors marketing reconciliation.

Working agreement (read CLAUDE.md too)

- Solo dev, commit directly to main, push when done, user pulls + tests
- No worktrees, no feature branches, no PRs
- Terse responses (caveman style). No preamble. Show diffs not walkthroughs.
- User has zero customers — never invent uplift %, CVR, recovered \$, testimonials

What this work is about

User asked: review the AI mode against the spec in `Resparq AI 5.26.26.pdf`, then build a roadmap of features. Roadmap converged on a multi-sprint plan to:

1. Fix real bugs (Starter gate)
2. Polish UX (progress bar theming, discount badge)
3. Build the 8 modal-design templates the marketing deck already promised but never existed in code
4. Wire templates to a manual picker + live preview
5. Then a "catch" component for dismissal recovery
6. Then AI integration (templateId as gene, meta-learning, Pro lift upsell)

Architecture decisions made (don't relitigate)

- **Modal templates = 8, not 10.** Cut "Free Shipping Bar" (economic conflict with merchant's existing shipping threshold app — kills their AOV strategy) and "Minimal Text" (too close to Classic Card). Final set: Classic Card, Top Banner, Bottom Sheet, Coupon Ticket, Split Hero, Timer-Front, Testimonial, Scratch Reveal.
- **Per-archetype variant cap.** Pro = 2 variants per archetype per segment (already in code at `variant-engine.js:911-926`). Enterprise = 20. Don't raise Pro — would collapse tier differentiation and starve bandits.
- **Cross-archetype template pooling within store.** When templateId becomes a gene, build a shop-level template posterior pooled across archetypes,

blended with cross-store meta-learning. 3-level hierarchical bandit:
archetype-specific → store-level pooled → cross-store meta. Weights
anneal with sample count.

- **Device personalization on Enterprise = priors, not segments.** Don't split iOS/Android/Windows into new segments — data fragmentation kills bandit performance. Add device-conditional posterior as a partial-pool prior layered on top of existing segment population.
- **Catch component:** mini-cart inline by default, cart-page banner if full cart page detected. Trigger = modal closed without claim. Lifespan = rest of session. Same theming pipeline as templates.
- **Queue tab (#7): DROPPED (2026-05-31).** Decided not to build. The queue had two sections — "Live now" (variants serving real traffic) + "On deck" (bred-but-warming + next-to-breed). Idea was Spotify-style drag-reorder. Rejected: the queue is bandit *output*, not a merchant playlist. Reordering Live-now overrides active traffic allocation and corrupts the posterior; even On-deck "next-to-breed" is recomputed each evolution cycle from fitness, so a manual order is an ephemeral nudge unless persisted as a pin. If merchant control is ever wanted, the compatible shape is **pin / exclude** (pin a variant to always-live, or veto/kill one) — NOT free reorder. For now the whole tab is cut.
- **Pro lift upsell (#5) = device-conditional upgrade nudge.** Enterprise gets device-conditional posteriors (offers personalized by device via partial-pool priors on existing segments — NOT new segments). Pro does not. The upsell analyzes the Pro store's OWN variant data, detects device cohorts behaving differently, and shows a UI card quantifying the lift left on the table → "upgrade to Enterprise." Same "detect but don't act" pattern as the existing promo-detection upsell (`ai-decision.server.js:132`). HARD RULE: the lift figure must come from the store's real device-split data; if there isn't enough, show a qualitative nudge, never a fabricated %.
- **No merchant context survey.** Earlier idea to ask onboarding questions about existing apps was rejected — kills the few-clicks install promise. Free Shipping Bar template is the only one with real economic conflict, and it was cut.

Sprint status

Sprint 0 — DONE (commit c6ff3a8)

- #6 Starter gate at `apps.exit-intent.api.ai-decision.jsx` — returns 403 upgradeRequired if `shopRecord.plan === 'starter'`. Storefront falls through to no-modal (existing graceful handling).
- #2 Progress bar theming — `cart-monitor.js` now sniffs theme CSS custom props for cart banner / qualification banner / mini-cart CTA.

Sprint 1 — DONE (commits 092384f, 24891da)

- #1 Tier 1 templates: Classic Card, Top Banner, Bottom Sheet, Coupon Ticket in `extensions/exit-intent-modal/assets/modal-templates.js`. Each is a self-contained renderer with same input contract. Registry exposed as `window.ResparqTemplates.{render, list, getThemeTokens, setThemeTokens}`.
- Dispatcher branch in `exit-intent-modal.js:createModal()` — uses template registry when `mode === 'manual' && templateId && ResparqTemplates` is loaded. AI mode keeps legacy `createModal` body (will be wired in Sprint 3).
- #4 Manual picker UI in `QuickSetupTab.jsx` with SVG thumbnails. Saves to `settings.manualTemplateId` in metafield. Hidden when AI mode active.
- #6 Live preview — JSX mirrors of each template in `SettingsPreview.jsx` (`ClassicCardPreview` / `TopBannerPreview` / `BottomSheetPreview` / `CouponTicketPreview`). Dispatch on `selectedLayout` prop. Re-renders on every state change.

Bug fix on Sprint 1 (commit 24891da)

- Storefront templates were transparent on some themes — theme-sniffing returned invalid colors. Fixed by adding `tokensFor(overrides)` merger that prefers merchant brand settings (`settings.brand*`) over auto-sniffed values. Background forced opaque via `isSafeOpaqueColor` guard, falls back to `#ffffff`.
- Discount amount didn't show in preview. Added `DiscountBadge` ("X% OFF" / "\$X OFF") above headline on Classic Card and Bottom Sheet, inline-prefix on Top Banner. Coupon Ticket already showed amount as hero. Liquid now exposes `discountPercentage` and `discountAmount` to JS.

Sprint 2 — DONE (commits ed3e205, 8596b0d, 6f4de29, 969a4db, 655fced, 2a49ff4)

- #1 Tier 2 templates: Split Hero, Timer-Front, Testimonial, Scratch Reveal in `modal-templates.js` (tier:2 in registry + `MODAL_LAYOUTS`). `getAvailableLayouts()` now filters `tier <= 2` — Tier 2 is live in picker.
- JSX previews added in `SettingsPreview.jsx` (`SplitHero` / `TimerFront` / `Testimonial` / `ScratchReveal`) + thumbnails in `QuickSetupTab.jsx`.

- #3 Catch component: already shipped pre-Sprint-2 in commit 27e29b1

(mini-cart inline + cart-page banner in `cart-monitor.js`, offer pill in `exit-intent-modal.js`). Sprint 2 only added catch framing copy ("Still want your <savings>?") — no duplicate surface built.

- Notes on the Tier 2 renderers:
- Testimonial = merchant copy + decorative 5 stars only (no fabricated names/stats — user has zero customers, never invent social proof).
- Scratch Reveal CTA stays clickable regardless of canvas scratch state.
- Timer-Front is deadline-driven (not a hardcoded countdown). See fixes.

Sprint 2 follow-up fixes (post-build)

- **Timer-Front countdown** (6f4de29): was hardcoded ~15min. Now deadline-driven via `props.timerEndsAt` → `modal.dataset.resparqTimerEndsAt`, hours-aware (shows hh:mm:ss when $\geq 1h$). Dispatcher seeds 24h at render; `showModal()` calls `syncTemplateTimerDeadline()` after `generateUniqueCode()` to reconcile to the real `offerExpiresAt`. Self-clears interval when modal leaves DOM.
- **Mini-cart catch line squished** (969a4db): was inserted as a flex sibling of the checkout button. Now a full-width block mounted above the cart footer via `insertMiniCartRow()` in `cart-monitor.js`.
- **Duplicate unique codes stacking** (655fced, 2a49ff4): unique-code mode minted a fresh combinable code each session; the days-long cart cookie kept old ones applied → 2-3 EXITINTE codes stacking = double/triple discount. Fix: `generateUniqueCode()` now reuses any already-applied exit code instead of minting. Detection in `findAppliedIssuedCode()` = exact match against `localStorage.exitIntentIssuedCodes` (tracked via `recordIssuedCode`, capped 20) PLUS prefix match (any applied code sharing a tracked code's PREFIX-segment, e.g. EXITINTE10-..., catches codes minted before tracking existed).
Limitation: a theme app extension CANNOT remove already-applied discount codes (Shopify owns that at checkout). Codes stacked before this fix need a one-time manual clear (X in checkout / empty cart). Going forward only one exit code ever lives on a cart. Root combinesWith stays `orderDiscounts:true` (`discount-codes.js`) so exit offers still stack with the store's own promos — left intact intentionally; the dedup is purely storefront-side.

Sprint 3 — SHIPPED (2026-06-01)

All Sprint 3 items complete. New commits this round:

```

5f44022 Add dev/preview learning-write guard
ba13698 Pro device-lift upsell (detect-but-don't-act) + dashboard card
9e7b3bc 3-level hierarchical template posterior (enterprise) + meta priors
812e772 Component Analysis: Modal Design leaderboard column
b09f870 Flip LIVE_AI_RENDER on
c1aa0e8 Timer-Front graceful no-discount degrade + top-banner nav inset

```

What landed:

- **Live AI render ON.** `LIVE_AI_RENDER = true` in `exit-intent-modal.js:22`.

AI mode lazy-renders the evolved `decision.templateId` through `renderTemplate`. Per-visit kill switch: `?resparqLiveAI=1` forces on, `?resparqLiveAI=0` forces legacy DOM-patch path. Timer-Front degrades gracefully when there's no discount (no "offer expires in" with no offer); top-banner insets below sticky theme headers (`bannerTopInset()`).

- **3-level hierarchical template posterior** (`app/utils/template-priors.js`):

cascading shrinkage meta → store-pooled → archetype-specific, weights anneal with sample count. Wired into `variant-engine.js` as a beta-sample multiplier, gated `enableTemplatePriors = enterprise` only (Pro's 2-variant cap barely spans the layout space). Empty until store clears `MIN_STORE_IMPRESSIONS=30`.

- **Component Analysis Modal Design column** (`app/variants._index.jsx`): 4th

leaderboard column, aggregates by `v.templateId`, shows layout name + description from `MODAL_LAYOUTS` (not copy text). Empty until variants accumulate impressions.

- **#5 Pro device-lift upsell** (`app/utils/device-lift-upsell.server.js` +

card in `app._index.jsx`): analyzes the Pro store's OWN device-split data, detects cohorts preferring different layouts, quantifies lift left on the table → "upgrade to Enterprise." HARD RULE honored: quantitative % only from real device data (gates: 60 impressions/device, 20/template-cell, ≥2 cohorts, ≥2 distinct winners, ≥5% lift); falls back to qualitative nudge; null if no divergence. NEVER fabricates a %.

- **Dev-data poisoning guard** (`app/utils/dev-shop-guard.server.js`): `dev/test`

stores + preview renders no longer write to the learning tables. Explicit allowlist (env `RESPARQ_DEV_SHOPS` comma-separated + hardcoded `exit-intent-test-2.myshopify.com`) plus per-request `isPreview` signal — all stores are `*.myshopify.com` so can't skip by suffix. Gates the decision endpoint's 5 write sites + the order webhook's 2 conversion writes.

Verification debt: Sprint 3 was lint/parse-checked and headless-validated but NOT yet run against a live store with the AI flag on. The posterior + upsell + Modal Design surfaces only populate once real impressions accumulate. Do a

real-theme eyeball of the live AI render before treating it as battle-tested.

Step 1 foundation — DONE (commit c6a69fb):

- `templateId` is now a gene on `Variant (@default("classic-card"))`, migration `20260531120000_add_template_id_to_variant` applied locally + committed.
- Gene pool: shared `TEMPLATE_IDS` (all 8 layouts) added to every archetype in `gene-pools.js` (cross-archetype pooling, per locked decision).
- Evolution engine wired in `variant-engine.js`: random create, diverse seed (spread evenly), crossover, mutation (`geneToPoolKey.templateId`), and the proven-gene seeding override all handle `templateId`.
- Meta-learning: `aggregate-gene-performance.js` aggregates `templateId` as a `geneType`, so cross-store `MetaLearningGene` populates it (covers original step 2). `determineBaseline` handles it via the generic string branch.
- Decision payload: `apps.exit-intent.api.ai-decision.jsx` now returns `decision.templateId` on both variant-bearing return shapes (discount + no-discount). Enterprise/variant path only — Pro `determineOffer` is rule-based.

Storefront render refactor — foundation DONE (commit ad01570; seams + preview harness):

- `exit-intent-modal.js` now has a single render pipeline. Extracted `buildTemplateProps(content)` (one source of truth for render props + themeOverrides; copy injected as props, never patched into DOM) and `renderTemplate(templateId, props)` (mounts, stamps legacy IDs, wires handlers, replaces any prior modal so AI can lazy re-render). Manual path `renderFromTemplateRegistry()` refactored to route through both — no behavior change.
- **Preview harness:** `?resparqPreview=<templateId>` renders any layout with representative AI-style copy and shows it immediately, bypassing the bandit, the dev-poisoned intervention threshold, and all triggers. THIS is how you verify all 8 layouts in a real theme without an exit/convert. Available ids logged to console. Use it to QA before enabling the live AI path.

Validation run — 2026-05-31 (headless, no live store):

- Ran Chrome headless + Node against a standalone harness that loads the real `modal-templates.js` and mirrors `buildTemplateProps / renderPreviewTemplate` sample props. All 8 layouts rendered correctly (screenshots eyeballed).
- Contract checks PASS for all 8: legacy IDs stamped (`#exit-intent-modal-overlay/-modal/-primary-cta/-secondary-cta`), close +

primary + secondary handles present, brand accent applied, discount code shown where applicable, `render ( )` returns the requested `templateId` (no silent fallback).

- Engine/cross-file PASS: `TEMPLATE_IDS === storefront registry ids` (no AI→renderer mismatch), all 5 archetype pools expose valid `templateIds`, random/diverse/crossover selection stays in-set (400 iters), Prisma migration applied, ai-decision returns `templateId` on both return shapes (lines 501/580/731).
 - Gap CONFIRMED (not a regression): `updateModalWithAI` still DOM-patches (`querySelector( 'h2' ), p, #modal-primary-cta`) and ignores `decision.templateId`. AI-chosen template ships in the payload but isn't rendered yet — this is exactly the "Live AI render" TODO below.
 - Minor cosmetic notes for later: top-banner concatenates 15% OFF — headline (long on mobile); testimonial uses the subhead as the quote. Both by design.
- Sprint 3 TODOs — ALL DONE:
- **~~Live AI render.~~** DONE (b09f870). Flag ON. Kill switch `?resparqLiveAI=0`.
 - **~~3-level hierarchical template posterior.~~** DONE (9e7b3bc), `app/utils/template-priors.js`, enterprise-gated.
 - **~~#5 Pro lift upsell.~~** DONE (ba13698), `device-lift-upsell.server.js`.
 - **~~Modal Design column on Component Analysis tab.~~** DONE (812e772).
 - **~~#7 Queue tab~~** — DROPPED 2026-05-31 (see architecture decision).

Next instance — where to pick up

Sprint 3 code is shipped but unproven on a live store. In priority order:

1. **Real-theme eyeball of the live AI render.** Flag is ON. Open a real store in AI mode, trigger an exit, confirm the evolved `decision.templateId` renders (not legacy DOM-patch). If anything looks wrong, `?resparqLiveAI=0` is the per-visit kill switch; flipping `LIVE_AI_RENDER` back to `false` in `exit-intent-modal.js:22` is the global one.

2. **Confirm the dev guard is working.** Test orders from `exit-intent-test-2.myshopify.com` (or whatever's in `RESPARQ_DEV_SHOPS`) should NOT create `VariantImpression / InterventionOutcome / InterventionThreshold` rows. Watch for the "[dev-guard] skipping learning write" console line.

3. **17-factors marketing reconciliation** (see Known issues) — still open.

4. **Let traffic accumulate**, then verify the posterior multiplier, the Modal

Design column, and the Pro device-lift card actually populate.

How the template system actually works

Storefront flow

```
Liquid snippet
  loads window.exitIntentSettings (includes templateId, brand colors, discount)
  loads modal-templates.js (registry, defines window.ResparqTemplates)
  loads exit-intent-modal.js
  loads cart-monitor.js

exit-intent-modal.js .init()
→ createModal()
  if (mode === 'manual' && templateId && ResparqTemplates)
    → renderFromTemplateRegistry()
      builds props from settings (headline, subhead, cta, amountText)
      builds themeOverrides from settings.brand* (merchant-controlled)
      calls ResparqTemplates.render(templateId, props)
      stamps IDs (#exit-intent-modal, #modal-primary-cta, #modal-secondary-cta)
        so legacy show/hide/handle code keeps working
      wires close/primary/secondary handlers
  else
    → legacy createModal body (the original ~280 lines, untouched)
```

Theme token resolution priority (modal-templates.js tokensFor)

override.primary	sniffed --color-button / --color-accent-1 / --color-foreground
override.primaryText	sniffed --color-button-text / --color-background
override.background	(must pass isSafeOpaqueColor) sniffed (same) '#ffffff'
override.foreground	sniffed --color-foreground / --color-base-text '#1a1a1a'
override.fontFamily	sniffed primary-button font system stack
borderRadius	always from sniffed primary button

Merchant brand settings ALWAYS win when set (≠ defaults). Auto-sniffing is

fallback for first-run / unconfigured stores.

Registry contract

Each template in modal-templates.js is render(props) → { overlay, modal, primaryCta, secondaryCta, closeButton }. Props always include:

```
{
  headline, subhead, cta, secondaryCta,
  code, amount, amountText,           // amountText is the human label "15%" / "$10"
  showSecondary, showPoweredBy,
  themeOverrides                       // optional merchant brand color override
}
```

To add a Tier 2 template:

1. Add a renderXxx(props) function in modal-templates.js following the same shape (use tokensFor(props.themeOverrides), makeCloseButton, makePrimaryButton, makeDiscountBadge, makePoweredBy helpers)
2. Register in the TEMPLATES object at the bottom of that file
3. Add entry to MODAL_LAYOUTS in app/utils/templates.js with tier: 2

4. `getAvailableLayouts()` filters by `tier <= 2` today — bump the cap when ready to expose a higher tier in the picker
5. Add a matching JSX preview component in `SettingsPreview.jsx` + register in the switch inside `ModalCard` component
6. Add a thumbnail case in `LayoutThumbnail (QuickSetupTab.jsx)`

Known issues / gotchas

- **No-intervention loop in dev — GUARD SHIPPED (5f44022).** Adaptive intervention threshold (`app/utils/intervention-threshold.server.js`) was poisoned by dev testing data. After 10 outcomes per score bucket, Thompson Sampling kicks in; if show arm accumulated 0-conversion impressions, AI permanently said "no intervention" (console spam "AI decided not to show a modal"). Now guarded by `app/utils/dev-shop-guard.server.js`: `isLearningWriteSkipped({ shopDomain, isPreview })` skips all writes to `InterventionThreshold / InterventionOutcome / VariantImpression` for allowlisted dev shops (`env RESPARQ_DEV_SHOPS` + hardcoded `exit-intent-test-2.myshopify.com`) and preview renders. Gates the decision endpoint (5 sites) + order webhook (2 conversion writes). To add a dev store: `set RESPARQ_DEV_SHOPS=foo.myshopify.com,bar.myshopify.com`.
- **AI mode + manual `templateId`.** AI mode currently ignores `templateId`. AI uses legacy `createModal` body and AI-decided copy. This is by design for Sprint 1 — Sprint 3 wires `templateId` into the bandit gene set.
- **Legacy `createModal` IDs.** Downstream code in `updateModalWithAI` and `closeModal` query selects `#exit-intent-modal`, `#modal-primary-cta`, `#modal-secondary-cta`. New templates MUST receive these IDs from the dispatcher (it stamps them). Don't rename.
- **Existing `MODAL_TEMPLATES` constant is copy presets, NOT layouts.** Naming collision risk. The new visual templates are `MODAL_LAYOUTS` in the same file (`app/utils/templates.js`). Keep distinct.
- **17 factors claim.** Spec/marketing says "17 factors" — code has ~15-16 signals in `ai-decision.server.js:185-286`. User hasn't decided whether to bump to 17 or change the marketing. Pending.

Open product questions

1. **Scratch Reveal — RESOLVED.** Shipped in Sprint 2 (commit `ed3e205`).

Canvas-based with CTA kept clickable regardless of scratch state.

2. **Catch component copy.** Should "Still want your discount?" copy be fixed or evolve via bandit? Default: fixed for now, defer to AI later.

3. **Theme app extension block** for true-native progress bar render. Not started. Deferred — current theming-via-sniff is good enough for now.

Key files

app/routes/apps.exit-intent.api.ai-decision.jsx	AI decision endpoint, plan gate
app/utls/ai-decision.server.js	Pro determineOffer + Enterprise AI
app/utls/variant-engine.js	Thompson sampling, evolution
app/utls/intervention-threshold.server.js	Adaptive show/skip threshold
app/utls/templates.js	MODAL_TEMPLATES (copy) + MODAL_LAYOUTS (visual)
app/components/settings/SettingsPreview.jsx	Live preview pane + layout previews
app/components/settings/tabs/QuickSetupTab.jsx	Manual picker UI + LayoutThumbnail
app/routes/app.settings.jsx	Settings page, state, action handler
extensions/exit-intent-modal/assets/modal-templates.js	Template registry + 4 renderers + helpers
extensions/exit-intent-modal/assets/exit-intent-modal.js	Storefront modal lifecycle (dispatcher + legacy)
extensions/exit-intent-modal/assets/cart-monitor.js	Progress bar + cart banners (theme-aware)
extensions/exit-intent-modal/snippets/exit-intent-modal.liquid	Asset loading + settings JSON

Recent commits

2a49ff4	Match exit codes by prefix when deduping to catch untracked stacked codes
655fced	Dedup unique discount codes to prevent stacking on persistent cart
969a4db	Fix squished mini-cart catch line in flex-footer themes
6f4de29	Fix Timer Front countdown to match real promotion window
8596b0d	Sprint 2: catch framing on dismissal-recovery surfaces
ed3e205	Sprint 2: 4 Tier 2 modal templates (Split Hero, Timer Front, Testimonial, Scratch Reveal)
27e29b1	Stack exit offers with store promos; stop auto-pausing AI during promos
24891da	Fix transparent storefront modal + add discount badge to preview/storefront
092384f	Sprint 1: 4 modal-layout templates + manual picker + live preview
c6ff3a8	Sprint 0: gate Starter from AI mode + native theming for cart progress bar

When user says "ready" or "go" next time

Next concrete work is **Sprint 3** (the bandit/AI integration — biggest chunk, touches DB schema + evolution engine + admin UI). Decisions are locked above ("Architecture decisions made — don't relitigate"). Suggested order:

1. **templateId as a gene** — Prisma migration adding templateId to Variant; wire into variant-engine.js (Thompson sampling / breeding) as a gene.

Foundation for everything else. Build the 3-level hierarchical template posterior (archetype → store-level pooled → cross-store meta), weights anneal with sample count.

2. **Populate MetaLearningGene with templateId** — so cross-store meta-learning has the new gene.

3. **Modal Design column on Component Analysis tab** — surfaces the new

templateId gene as a fourth leaderboard column in app/routes/app.variants._index.jsx. Aggregate by v.templateId (mirror byHeadline/bySubhead/byCTA at ~line 733), emit performance.templates, widen grid '1fr 1fr 1fr' → 4 cols at ~line 1448. Card shows layout name/thumbnail (from MODAL_LAYOUTS), not copy text — needs a ComponentCard type="template" branch. Gated on step 1 (no templateId on Variant until the migration lands).

4. **#5 Pro lift upsell** — device-conditional posterior as a partial-pool prior layered on existing segment population (NOT new segments — data fragmentation kills bandits). Plus the UI upsell card.

(Queue tab #7 dropped 2026-05-31 — see Architecture decisions.)

Don't restart the architectural conversation — decisions above are locked.

Open carry-over items (post-Sprint-3)

- **~~Dev-data poisoning guard~~** — SHIPPED (5f44022). See Known issues.
- **17 factors claim**: code has ~15-16 signals; marketing says 17. STILL

UNRESOLVED. Either bump the code to 17 signals

(ai-decision.server.js:185-286) or change the marketing. User hasn't decided.

- **Catch copy bandit / native theme block** — see "Open product questions".

Both deferred. Scratch Reveal shipped in Sprint 2.

Session 2026-06-05 — Signal wiring audit + Pro/Enterprise unification plan

This session did two things: (1) resolved the 17-factors claim and audited/fixed the intent-signal wiring, and (2) discovered that Pro and Enterprise are two forked decision engines and produced a plan to unify them. ****The unification is NOT built yet — it is the next instance's main job.**** The concrete numbers (propensity bands, discount curve, margin floor) are specified below so the next instance does not have to re-derive them.

STATUS — read first

- The signal-wiring fixes below are ****applied as local edits but NOT yet committed/pushed**** (unless a commit was made after this doc was written —

check `git log`). The next instance MUST `git pull` and confirm these changes are present, or they will be working against stale code.

- The unification is a **plan only**. No unification code has been written.

17 factors — RESOLVED (was unresolved in prior handoff)

Decision: bump the code to 17 signals (not change marketing). Added two scoring contributions in `determineOffer (ai-decision.server.js)`:

- `abandonmentCount` (repeat cart abandonment): $\geq 1 +15$, $\geq 3 +10$.
- This brought the Pro intent-score block to 17 distinct factors. Full table of all 17 (source + weight + rationale) was added to `AI_TECHNICAL_ARCHITECTURE.md` ("The 17 Intent-Scoring Factors").

Signal-wiring audit — 5 defects found and fixed

Audited all 17 factors end-to-end (collected → sent/enriched → scored). 12 were fine. Five were broken and were fixed this session:

- **#10 hasAbandonedBefore** — was DEAD. The `abandonedCart=true` cookie it reads was never set anywhere. Always false.
- **#11 abandonmentCount** — was DEAD. The `exitIntentAbandonments` localStorage counter it reads was never written. Always 0.
 - Fix for #10 + #11: added `trackCartAbandonment()` in `exit-intent-modal.js` (called in `init`). Detects abandonment via **next-session cart carryover** — if a new session starts with items still in the cart from a prior session, the prior session abandoned. Sets the cookie + increments the counter, guarded so the same physical cart (by `cartFirstItemTimestamp`) is counted once. Conversion is auto-excluded because checkout empties the cart, so a converted cart never carries over. Chosen over `beforeunload` (fires on internal nav, can't tell "left site" from "went to checkout & bought").
- **#14 productDwellTime** — read bug. `getProductDwellTime()` returned 0 unless the visitor was CURRENTLY on a product page, discarding all accumulated dwell whenever the exit happened on cart/checkout/collection (the common case). Fixed to return the accumulated session total regardless of current page.
- **#15 purchaseHistoryCount** — structurally inert in Pro. The 17-factor score lives in `determineOffer` (Pro only), but the Pro storefront path (`getAIDecision`) sends raw signals with no enrichment, so this was always

undefined. Fixed with **server-side enrichment** in the `ai-decision` endpoint:
reads the app-proxy `logged_in_customer_id`, queries `customer.numberOfOrders`,
sets `signals.purchaseHistoryCount` before the decision. Works for both tiers,
no extra storefront round-trip. Guests correctly get 0.

- **enrich-signals.jsx (Enterprise path) double bug** — it read `response.data?.customer` without `await response.json()` (so it silently got nothing) AND queried the removed `ordersCount` field (gone in API 2026-01).
Fixed both: `await response.json() + numberOfOrders`. This had made `purchaseHistoryCount` inert on the Enterprise propensity path too.

Files touched this session:

- `extensions/exit-intent-modal/assets/exit-intent-modal.js` (`trackCartAbandonment`, `getProductDwellTime`)
- `app/routes/apps.exit-intent.api.ai-decision.jsx` (`purchaseHistoryCount` enrichment)
- `app/routes/apps.exit-intent.api.enrich-signals.jsx` (`.json() + numberOfOrders`)
- `app/utils/ai-decision.server.js` (`abandonmentCount` scoring)
- `AI_TECHNICAL_ARCHITECTURE.md` (17-factor table)

THE BIG FINDING — Pro and Enterprise are a hard fork

There are two learning layers and they are in OPPOSITE states:

- **Layer 1 — variant/template bandit** (`variant-engine.js`): already unified,
param-driven. Enterprise differs only by parameters (variant cap Pro 2 /
Enterprise 20, device-conditional priors). This is the desired shape. Leave it.
- **Layer 2 — offer / show-decision engine** (`ai-decision.server.js`): a HARD
FORK. `determineOffer()` (Pro) and `enterpriseAI()` (Enterprise) are entirely
separate functions. This is what must be unified.

Concrete divergences in Layer 2:

- **Show/skip metric differs.** Pro feeds an additive 17-factor score
(unbounded, ~ -50..+200) into `shouldIntervene`; Enterprise feeds a 0-100
`propensityScore` (a different, log-scaled formula in `enrich-signals`) into
the same function. Same bandit, two incompatible scales → split learning.
- **Hard overrides differ.** Pro only has SKIP overrides. Enterprise FORCE-SHOWS
via 4 dedicated offer builders (`discount-seeker`, `checkout-recovery`,
`price-sensitive`, `stale-cart`) that Pro lacks entirely.
- **Offer math differs.** Pro: `score` → 5-25%, fixed-vs-%, margin-capped via
`capDiscountForProfitability`. Enterprise: propensity tiers (>75 / 40-75 / <40)

+ time-of-day modulation, and **NO margin cap at all** (biggest margin risk in the codebase).

- **Timing differs.** Enterprise returns timing; Pro relies on storefront evolved trigger genes.

Already shared (keep): detectFunnelGoalFromSignals, the shouldIntervene adaptive-threshold call, and device segmentation at the threshold (mobile/desktop/all — both tiers already do this).

Side findings:

- determineOffer's plan === 'enterprise' promo branch (lines ~144-182) is DEAD CODE — Enterprise never calls determineOffer. Delete during unification.
- Margin-capping is Pro-only today; Enterprise must gain it (see margin floor).

UNIFICATION PLAN (Layer 2) — the next instance's main task

Goal stated by the user: maximize conversion lift that is ****noticeable in the store's Resparq dashboard****, WITHOUT throwing big offers at every add-to-cart (a conversion that loses money on the order is worthless). Collapse determineOffer + enterpriseAI into ONE engine where tier is pure config.

Target shape:

```
decideOffer(signals, ctx) -> { show, type, amount, threshold, timing,
                             triggerReason, reasoning, confidence }
ctx = { plan, aggression, cartValue, shopId, tierProfile, marginFloor, testMode }
```

Four shared stages (each currently duplicated/forked):

- **Stage 1 — One scoring metric: unified propensity P in [0,100]** = P(convert WITHOUT an offer). Promote calculatePropensity (currently in enrich-signals.jsx, 23-signal log-scaled) into a shared propensity.server.js used by both tiers AND the endpoint. RETIRE Pro's additive score block; the 17 factors fold into propensity. High P → little/no offer; low P → bigger offer.
- **Stage 2 — One show/skip decision.** Both tiers feed the SAME propensity into the SAME shouldIntervene(db, shopId, P, segment). One ordered hard-override list (force-show: failedCoupon, checkoutExit, hesitation>1, staleCart>60; force-skip: accidental visit, P>=85). Consistent triggerReason for both.
- **Stage 3 — One offer constructor, margin-aware** (see curve + floor below).

The 4 Enterprise trigger builders become presets available to BOTH tiers.

Revenue (AOV threshold) vs conversion (direct discount) modes shared.

Time-of-day modulation available to both.

- **Stage 4 — Margin guardrail, always-on.** Apply the margin floor to EVERY

path (Enterprise currently skips it). Keep checkBudget and the existing holdout group (isHoldout) so dashboard "recovered revenue" is true incremental lift, not inflated.

Tier becomes a profile, not a fork:

- variantCap: Pro 2 / Enterprise 20 (more modals → faster learning)
- explorationRate / annealing: Enterprise more aggressive (learns faster)
- deviceConditioning: Pro = segment threshold only; Enterprise = device-conditional

offer PRIORS (partial-pool, layered on the unified offer — NOT new segments, per locked decision)

- timingControl: Pro = trigger genes; Enterprise = engine emits timing
- promoControl: Pro = detect-only (upsell); Enterprise = auto-optimize
- marginFloor: enforced for both

THE NUMBERS — propensity, discount curve, margin floor (USE THESE)

These were chosen so a store sees a real, attributable lift in the dashboard while no single offer can turn an order unprofitable. Anchored to the existing capDiscountForProfitability assumption (~40% avg margin, give away at most half of it), made configurable.

Unified propensity P in [0,100] = probability the visitor converts with NO offer.

Discount curve (conversion mode), pre-margin-clamp:

```
D_MIN = 5      // percent - below this, do NOT discount; show no-discount/announce
D_MAX = 25     // percent - absolute ceiling on any single exit offer
P_LO = 20
P_HI = 80
d_raw(P) = clamp( D_MIN + (D_MAX - D_MIN) * (P_HI - P) / (P_HI - P_LO), D_MIN, D_MAX )
d_curve = d_raw * (aggression / 5) // merchant dial 0-10, default 5 -> 1.0x
```

Margin floor (hard guardrail, always-on, applied after the curve):

```
assumedGrossMargin = settings.assumedGrossMargin ?? 0.40 // overridable per store
MAX_MARGIN_SHARE = 0.50 // an offer may consume at most half the gross margin
MARGIN_FLOOR = 0.20 // post-discount gross margin must stay >= 20%

shareCap = MAX_MARGIN_SHARE * assumedGrossMargin // 0.40 -> 20%
floorCap = 1 - (1 - assumedGrossMargin) / (1 - MARGIN_FLOOR) // 0.40 -> 25%
aggrCap = 10 + aggression * 1.5 // existing capDiscountForProfitability: 10-25%
final_d = min(d_curve, shareCap, floorCap, aggrCap, D_MAX)
if (final_d < D_MIN) -> no-discount / announcement modal (zero margin cost)
```

Consolidate this into capDiscountForProfitability (extend it to take assumedGrossMargin; keep the aggression term it already has).

CRITICAL — aggression is the MERCHANT'S dial, not an AI-learned value. The store

sets "Promo Aggression" (0-10) in the Admin (AISettingsTab.jsx, persisted in the shop settings metafield -> Shop.aggression). The decision endpoint reads it from settings.aggression and passes it into the engine; the AI must NEVER override it upward. It appears in TWO places and both must keep listening to the store value:

1. $d_curve = d_raw * (aggression / 5)$ — scales the whole curve. 0 = AI shows no discounts at all (announce-only); 5 = neutral 1.0x; 10 = 2.0x toward D_MAX.

2. $aggrCap = 10 + aggression * 1.5$ — the merchant's hard ceiling, still inside the margin floor. The AI may only move the offer DOWN from this cap, never up.

Per-segment/Bayesian learning tunes WHICH carts get discounted and the propensity estimate — it does not get to exceed the merchant's aggression ceiling. aggression=0 must short-circuit to a no-discount modal on every path.

Worked examples (assumedGrossMargin 0.40, aggression 5 -> aggrCap 17.5%, shareCap 20%):

```
P >= 85  -> announce only, $0 margin spent (they'll convert anyway)
P = 75   -> d_raw 6.7%  -> ~7%
P = 60   -> d_raw 11.7% -> ~12%
P = 45   -> d_raw 16.7% -> ~17%
P = 30   -> d_raw 21.7% -> clamped to 17.5% (aggrCap)
P = 20   -> d_raw 25%   -> clamped to 17.5% (aggrCap)
```

So default stores: hot carts 0-7%, warm ~12%, cool/cold capped ~17.5%. Merchant raising aggression lifts the ceiling toward 25% but the margin caps still bind first on thin-margin stores.

Why these numbers (so the next instance does not relitigate):

- **D_MIN 5%:** below ~5% an offer is ignorable and won't lift claims, so it produces no visible dashboard signal — better to show a no-discount/announce modal (captures the visitor, spends zero margin).
- **D_MAX 25%:** matches the historical ceiling; at 40% assumed margin a 25% discount still leaves ~33% post-discount margin — safe.
- **P_LO 20 / P_HI 80:** concentrates real discounts in the band where the modal is actually causal. At $P \geq 85$ the customer converts anyway, so discounting there burns margin AND pollutes attribution — skip/announce. The curve tapers to near-zero by $P=80$.
- **MAX_MARGIN_SHARE 0.50 + MARGIN_FLOOR 0.20:** even a worst-case thin-margin store keeps a positive, meaningful per-order profit; the guardrail binds before any offer makes the order a money-loser. This is the "don't decimate margin" requirement, enforced structurally.

How this makes value VISIBLE in the dashboard (the user's explicit ask):

- Concentrating spend on carts that actually need it raises **incremental** conversion per margin dollar — the recovered-revenue figure is both larger and

credible.

- Showing across the mid/low propensity band (not skipping everyone) gives a visible VOLUME of recovered carts.
- A $\geq 5\%$ minimum effective offer means claimed offers actually convert.
- Attribute everything against the existing 5% holdout (isHoldout) so "recovered revenue" is true lift, not inflated — a number the merchant trusts.

Migration (safe — live AI flag is ON, but ZERO customers so no real data to lose)

1. **Phase A** — Extract computePropensity + constructOffer + unified overrides into a core. Keep determineOffer/enterpriseAI as thin wrappers calling the core with a tierProfile. Zero caller change.

2. **Phase B** — Point the endpoint at decideOffer directly. Delete the forks and the dead determineOffer enterprise promo branch.

3. **Phase C** — RESET the adaptive-threshold buckets. Pro moves from additive score to propensity, so existing InterventionThreshold rows are on the wrong scale. Zero customers = ideal time to reset.

4. **Phase D** — Verify via the ?resparqPreview=<templateId> harness + a real-store eyeball (folds into the outstanding Sprint 3 verification debt).

Tests to add:

- **Golden-master:** snapshot current Pro + Enterprise outputs over a signal-fixture matrix; assert intended parity/improvement post-refactor.
- **Margin invariant:** randomized carts x offers -> assert post-discount margin $\geq$ MARGIN_FLOOR on EVERY code path. This is the regression guard for the "don't lose money" requirement.

Risks:

- Threshold metric scale change -> must reset bandit buckets (Phase C).
- Adding the margin cap to Enterprise shrinks some Enterprise offers -> slightly lower conversion on those, BY DESIGN (the conversion-vs-margin tradeoff).
- Time-of-day modulation is new behavior for Pro.

Don't relitigate the locked architecture decisions earlier in this doc; the unification implements them, it does not change them.